

Can we declare an abstract class with zero abstract methods?
Yes, but it is not common. However we can use this to enforce inheritance.

7.6.25

Abstraction → Interface

Q4

interface Language

```
{  
    void translate();  
}
```

class English implements Language

```
{  
    @Override  
    public void translate()  
    {  
        s.o.pln("English");  
    }  
}
```

class kannada implements Language

```
{  
    @Override  
    public void translate()  
    {  
        s.o.pln("kannada");  
    }  
}
```

class hindi implements Language

```
{  
    @Override  
    public void translate()  
    {  
        s.o.pln("Hindi");  
    }  
}
```

class GoogleTranslator

```
{  
    static void TranslateTo(Language l1, Language l2)
```

```
{  
        if (l1 != null && l2 != null)  
        {  
            l1.translate();  
            l2.translate();  
        }  
    }
```

public class Mainclass

```
{  
    p.s.v.m.c)  
    {  
        English e = new English();  
        Kannada k = new Kannada();  
        Hindi h = new Hindi();
```

```
        GoogleTranslator.TranslateTo(e, k);
```

```
        GoogleTranslator.TranslateTo(k, h);
```

```
        GoogleTranslator.TranslateTo(h, e);  
    }  
}
```


Q. 2

```
interface steam
{
    void test();
}

interface dream extends steam
{
    void disp();
}
```

```
class cream implements Dream
```

```
{
    @Override
```

```
void test()
{
```

```
}
```

```
@Override
```

```
void disp()
{
```

```
}
```

```
}

public class Mainclass
```

```
{
    D.S.V.M.C()
{
```

```
cream c = new cream()
```

```
c.test();
```

```
c.disp();
```

```
}
```

```
}
```

Definition - abstraction (Interface)

- we can declare an interface in java using the `interface` keyword.

keyword.

- Inside the interface we can declare data members which are `public`, `static` and `final` as well as members function which are `public` and `abstract`.
- we cannot create an object for an interface.
- we can access an properties of an interface by creating and implementation class and overriding the abstract methods.

Contract/Agreement.

- Whenever a concrete class implements a interface it is mandatory to override the acquired abstract methods
- A single class can provide implementation for more than one interface simultaneously.
- A class can acquire properties of an interface using implements keyword.
- An interface is a pure abstract body and helps to achieve 100% abstraction.



Abstract class

- It is a partial abstract body
- Declared using abstract keyword
- no restrictions on data members
- we can acquire using extends keywords
- Multiple inheritance is not allowed
- will have constructor
- will contain methods of object class

Interface.

- It is a pure abstract body
- Declared using Interface keyword.
- all member functions are public and abstract
- we can acquire using implements keyword.
- Multiple inheritance is allowed.
- will not have constructor.
- will not contain the methods of object class.

NOTES

- Whenever a class is implementing 2 or more different interfaces which contain abstract method having same declaration it is sufficient if we override only once.

```
eg) interface A
{
    void test();
}
```

```
interface B
{
    void test();
}
```

class C implements A, B

```
{
    @Override
    void test()
}
```


- When a class acquires from a class and interface simultaneously if the inherited concrete methods match with the abstract methods of interface it is not necessary to override

eg)

```

class P
{
    public void play()
    {
        s.o.pln("Executing play()...");
    }
}

interface Q
{
    void play();
}

class R extends P implements Q
{
}

public class MainClass11
{
    P s = v.m()
    {
        R obj = new R();
        obj.play();
    }
}

```

NOTE

*

class
↑ extends
class

interface
↑ implements
interface

interface
↑ implements
class

class
↑ ~~illegal~~
interface

QUESTION

1. Why implement is not considered as real inheritance.

When we acquire the property of an interface using into a class using implements keyword, there is no code reusability.

Hence it is not considered as a real inheritance.

Q.

Does

Java support multiple inheritance?

No, it will not support for class

Yes, it will support for interface.

(or)

I want Yes or No : **No**